The Campus of Tomorrow

Higher Colleges of Technology

كليات التقنية العليا

CDS 2413- PROGRAMMING FOR DATA SCIENCE

Week 11 - 12

Week 12	CLO 3: Create scripts to solve business problems using data analysis techniques such as classification, clustering, optimization, and recommendation

Document Revision CONTROL (DRC)

Version	Author	Effective Date	Change Description	DRC No
1.0	Mohammed Abdul Rahim	2/05/2023	Define the first version	001

X.Y
X: Major change
Y: minor change

Lecture Outline

- Clustering
- Types of Clustering
- Theory behind Clustering
- Python Programming for Clustering

LECTURE OBJECTIVES

At the end of the lecture, the student should be able to:

- **Understand** the function of **cluster analysis**.
- **Identify** the various forms of **cluster analysis**.
- **Investigate** how the **clustering functions**.
- **Write** cluster analysis **programming** in **Python**.

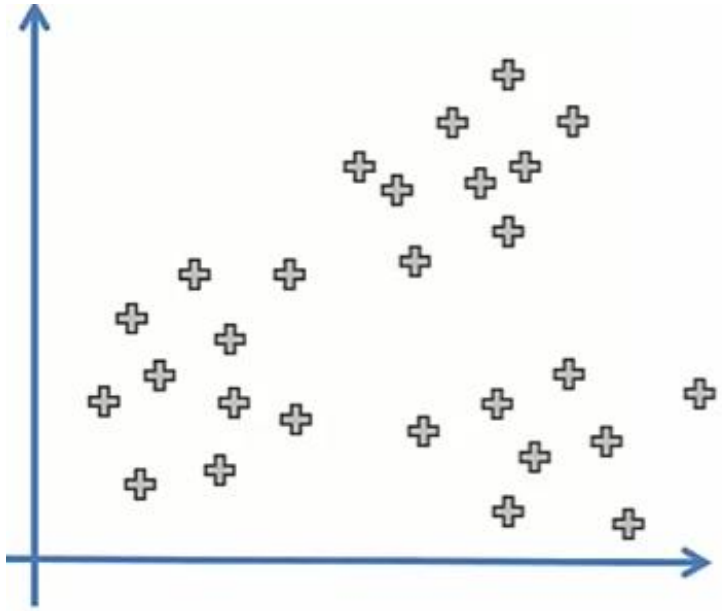
Hierarchical Clustering

Hierarchical Clustering

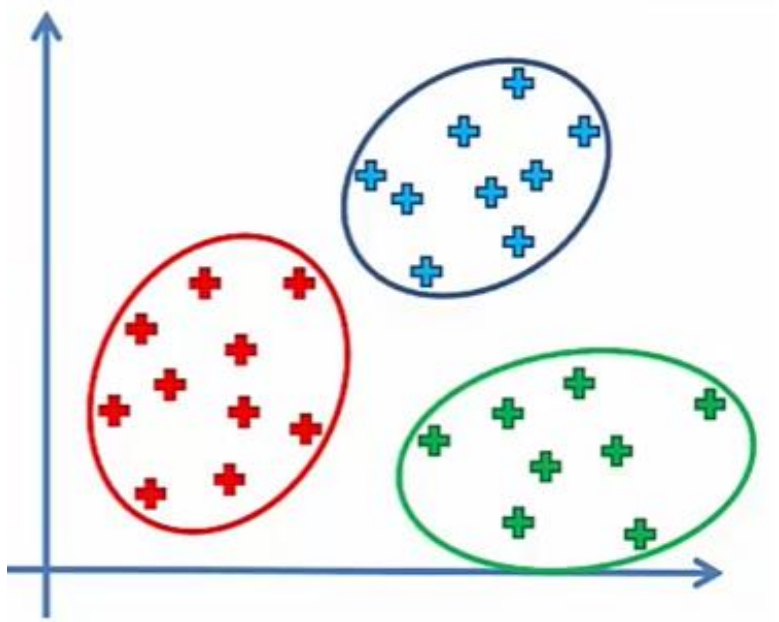
- Hierarchical clustering, also known as hierarchical cluster analysis (**HCA**),
- Is an unsupervised machine learning algorithm
- It groups similar data points into a hierarchical tree-like structure called a **dendrogram**.
- It works by iteratively merging or splitting clusters based on a similarity measure until a stopping criterion is reached.

Hierarchical Clustering

Before Hierarchical Clustering



After Hierarchical Clustering



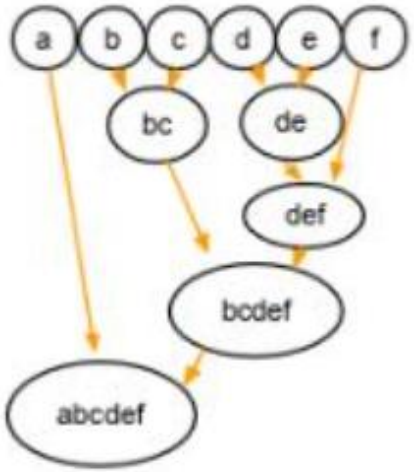
Hierarchical Clustering

- Hierarchical clustering uses a tree-like structure

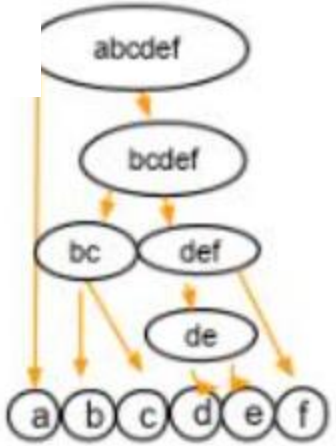
- Hierarchical clustering is further subdivided into:
  - Agglomerative clustering
  - Divisive clustering

Hierarchical Clustering

- **Agglomerative clustering:**
  - A bottom-up approach where each element starts as a separate cluster and merges into larger clusters.

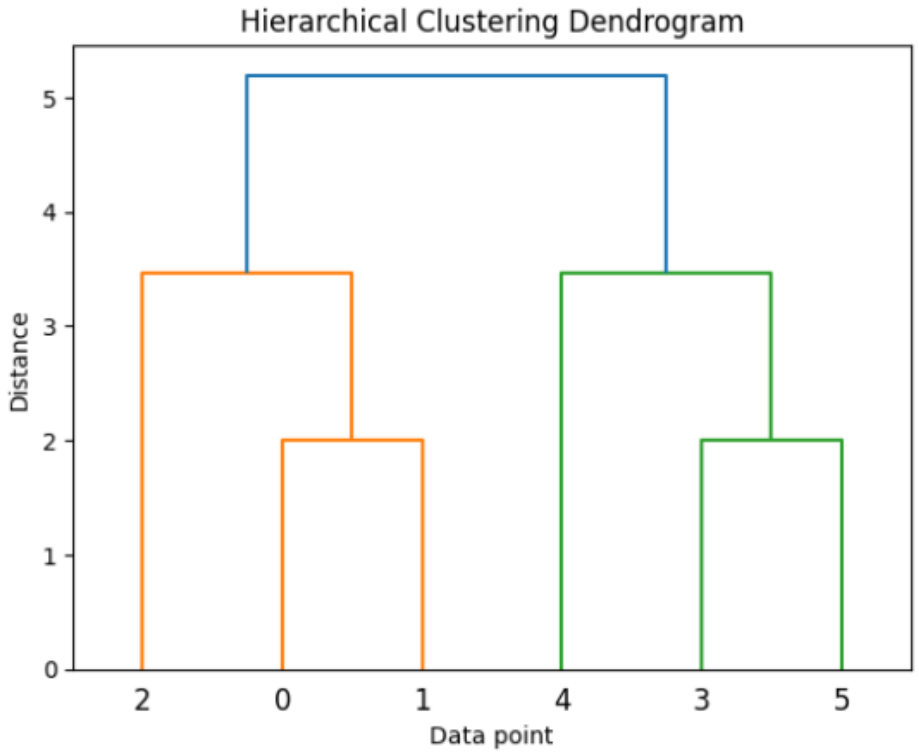


- **Divisive clustering:**
  - A top-down approach where the whole set is divided into smaller clusters.



Hierarchical Clustering – Agglomerative

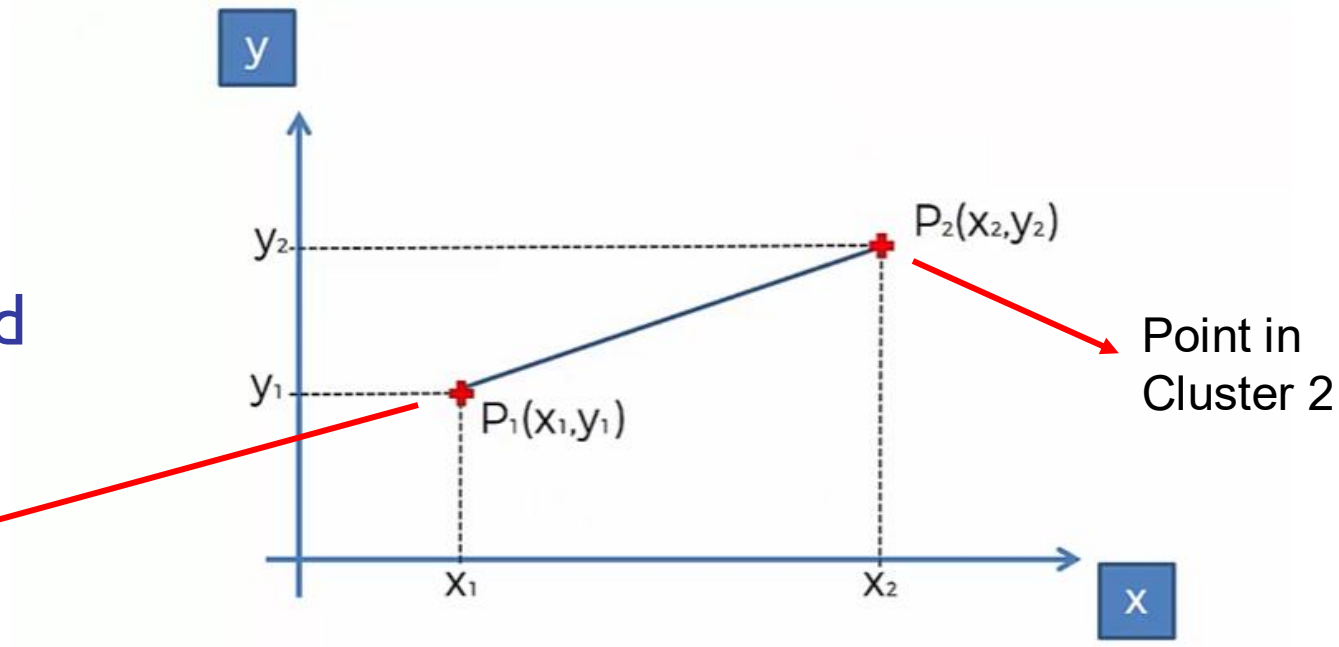
- **Agglomerative clustering** is the most common type of hierarchical clustering.
- It is also known as **AGNES** (**Agglomerative Nesting**).
- The algorithm starts by treating each object as a **singleton cluster** and then **merges pairs** of clusters until one big cluster remains.
- The result is a **dendrogram**.



Hierarchical Clustering Dendrogram

Hierarchical Clustering – Agglomerative

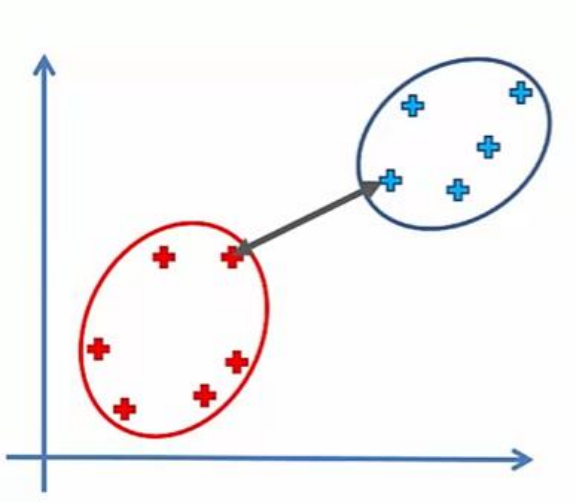
- Distance between the cluster is calculated by Euclidean method
- Shorter distance is chosen and clusters are merged



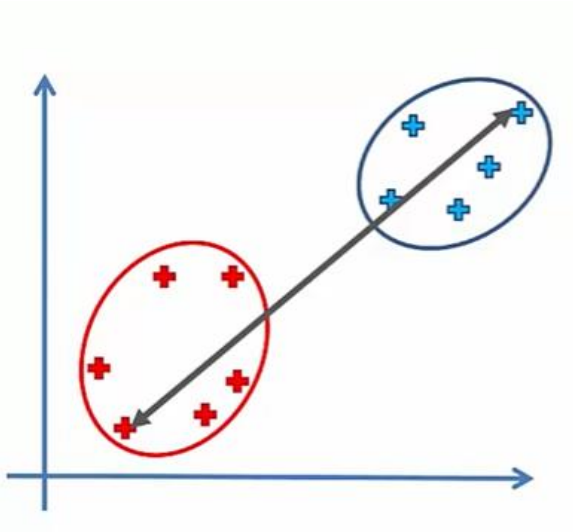
Euclidean Distance between P_1 and $P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Hierarchical Clustering – Agglomerative

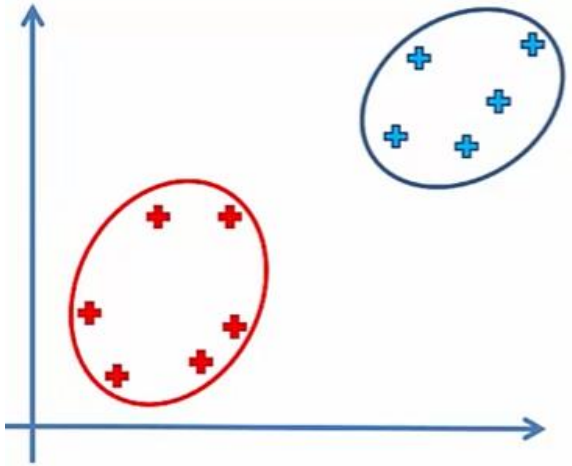
- Different options can be chosen to merge the cluster



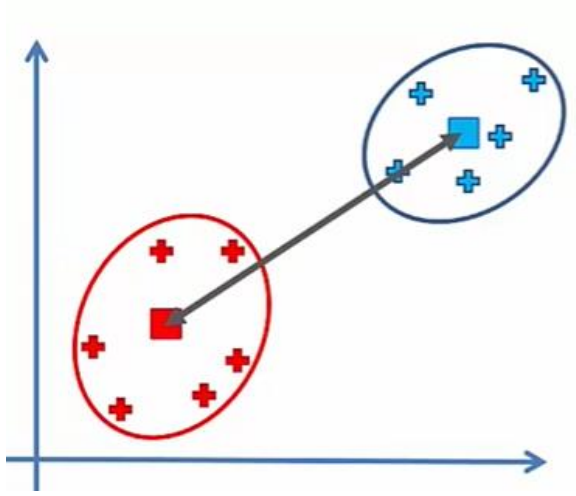
Option 1: Closest Points



Option 2: Furthest Points



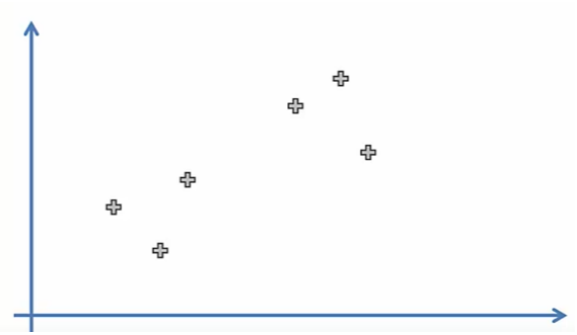
Option 3: Average Points



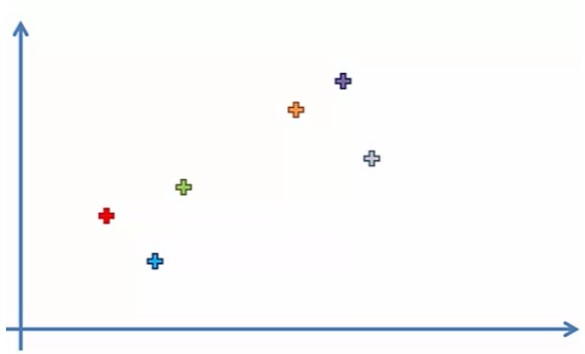
Option 4: Distance between the centroids

Hierarchical Clustering – Agglomerative

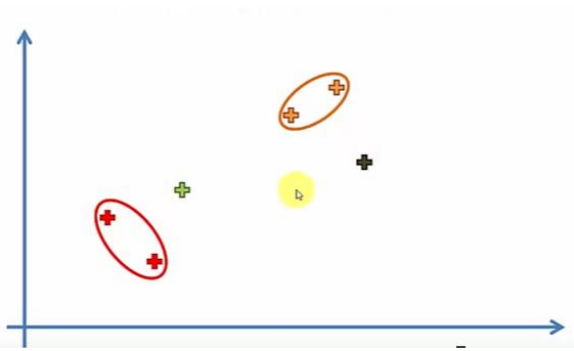
- Consider 6 data points. Apply Agglomerative techniques to merge the cluster



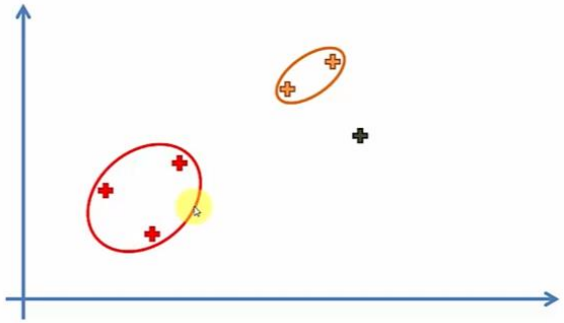
Six data points



Step 1: Make each data point as a cluster

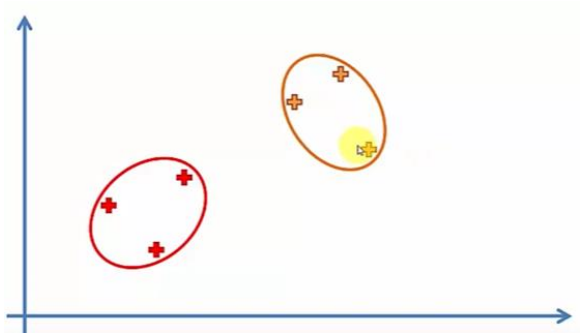


Step 2: find the distance between the cluster using Euclidian method. Take two clusters which are closer and merge

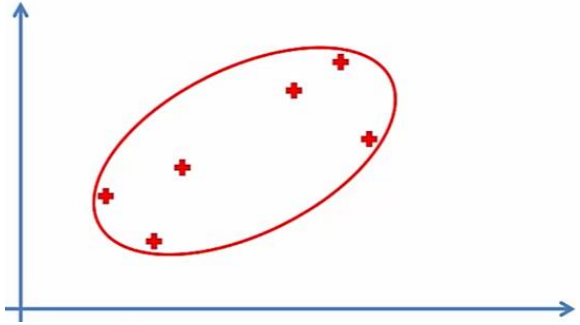


Step 3: repeat the step

Hierarchical Clustering – Agglomerative



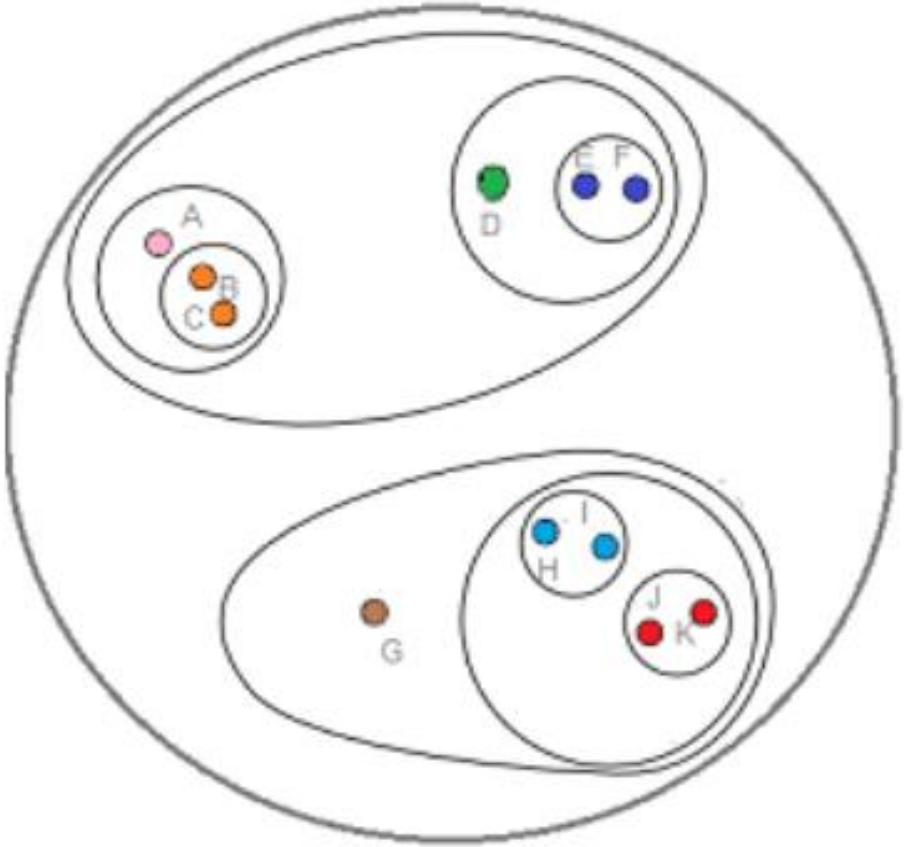
Step 3: repeat the step

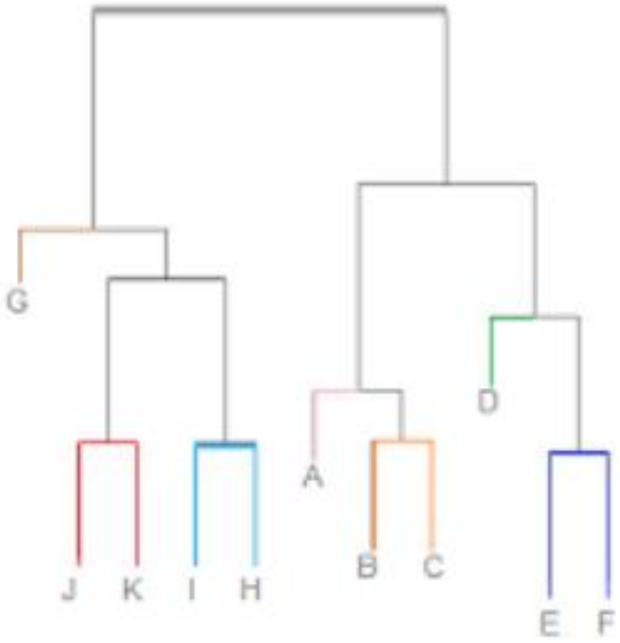


Step 4: Finally, form a single cluster

Hierarchical Clustering – Agglomerative

- For each cluster, create a Dendrogram
- Dendrogram is created to find the optimal number of clusters required for the given dataset





Build, Train and Find the Optimal Number of Clusters using Hierarchical Clustering– Case Study

Hierarchical Clustering - Dataset

- Dataset used Mall_Customer.CSV
- From the dataset, we can infer that we can't predict anything but can find some groups or patterns of the customer
- So we can use clustering algorithm instead of regression or classification algorithm

Customer	Genre	Age	Annual Income (k$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13

K-Means Clustering – Steps in Python

1. Import Libraries:

Python

```python
import pandas as pd
import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import AgglomerativeClustering
```

2. Prepare Data:

Load the data into a pandas DataFrame or NumPy array and perform any necessary preprocessing.

K-Means Clustering – Steps in Python

3. Compute Linkage Matrix:

Python

```
linkage_matrix = linkage(data, method='ward')  # Choose linkage method
```

4. Create Dendrogram:

Python

```
dendrogram(linkage_matrix, labels=data_labels)  # Optional: Include data labels
plt.xlabel('Cluster ID')
plt.ylabel('Distance')
plt.title('Dendrogram')
plt.show()
```

K-Means Clustering – Steps in Python

5. Perform Agglomerative Clustering:

Python

```
clustering = AgglomerativeClustering(n_clusters=desired_num_clusters, linkage='ward')
cluster_labels = clustering.fit_predict(data)
```

6. Analyze Clusters:

Use cluster labels to analyze the distribution of data points within each cluster.

Hierarchical Clustering– Steps in Python

Steps

1. Importing the libraries
2. Importing the dataset
3. Using the dendrogram to find the optimal number of clusters
4. Training the Hierarchical Clustering model on the dataset
5. Visualising the clusters

Hierarchical Clustering – Steps in Python

Importing the libraries

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Importing the dataset

In cluster analysis, we are not predicting any values. Therefore, separating the dataset's variables into dependent and independent variables is unnecessary.

```
[ ] dataset = pd.read_csv('Mall_Customers.csv')
    X = dataset.iloc[:, [3, 4]].values
```

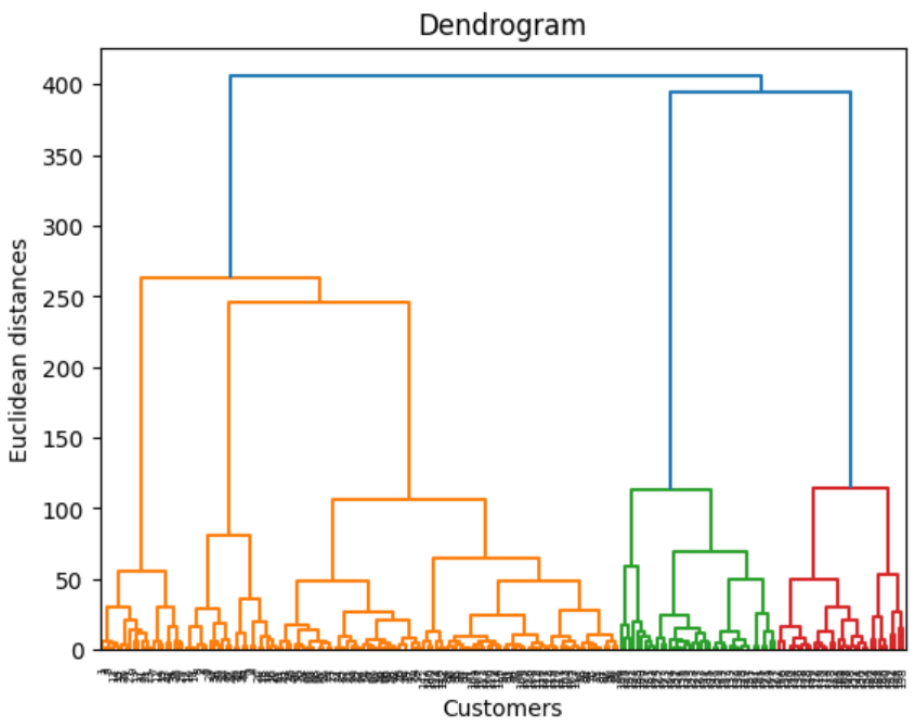
All rows

We must visualize the cluster at the conclusion of the analysis, which necessitates two columns (X-axis, Y-axis). Therefore, ignore the unimportant field and select the correct column (annual income - 3, expenditure score - 4).

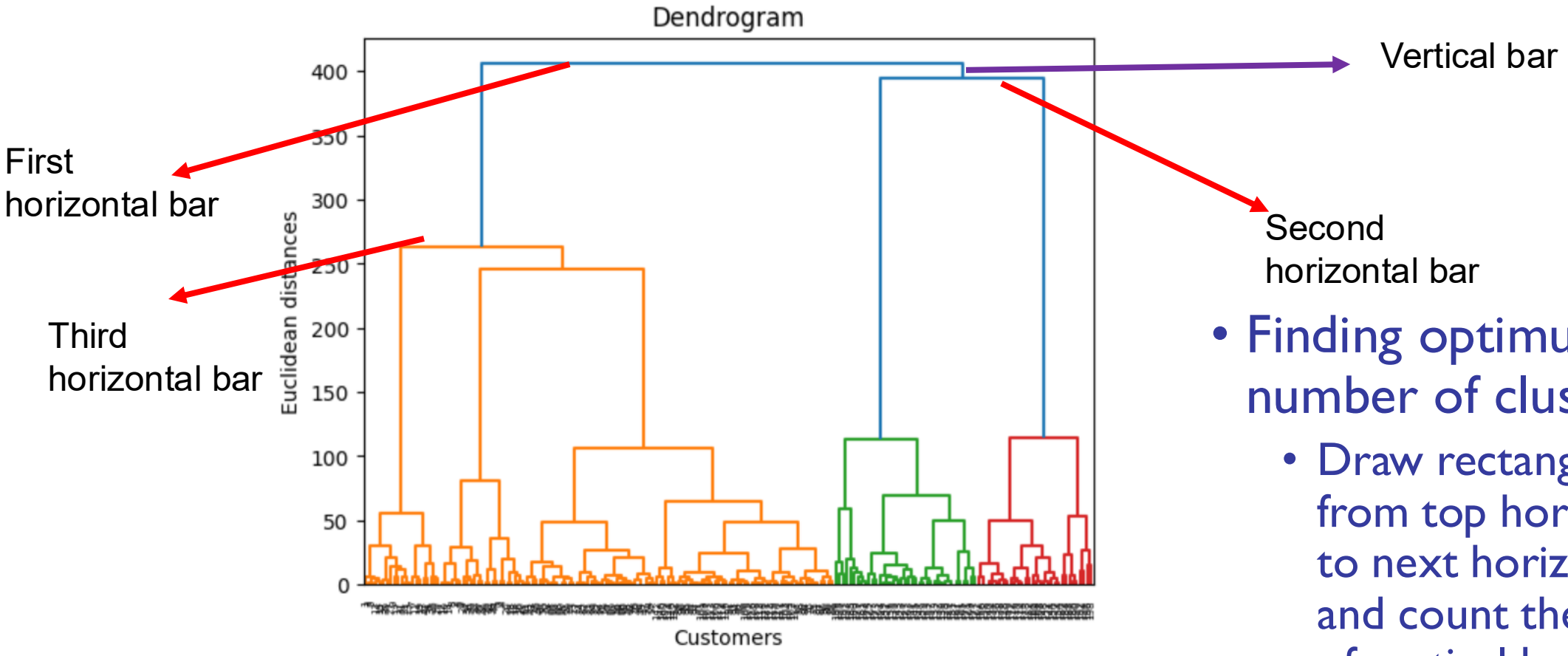
Hierarchical Clustering – Steps in Python

Using the dendrogram to find the optimal number of clusters

```
[3] import scipy.cluster.hierarchy as sch
    dendrogram = sch.dendrogram(sch.linkage(X, method = 'ward'))
    plt.title('Dendrogram')
    plt.xlabel('Customers')
    plt.ylabel('Euclidean distances')
    plt.show()
```



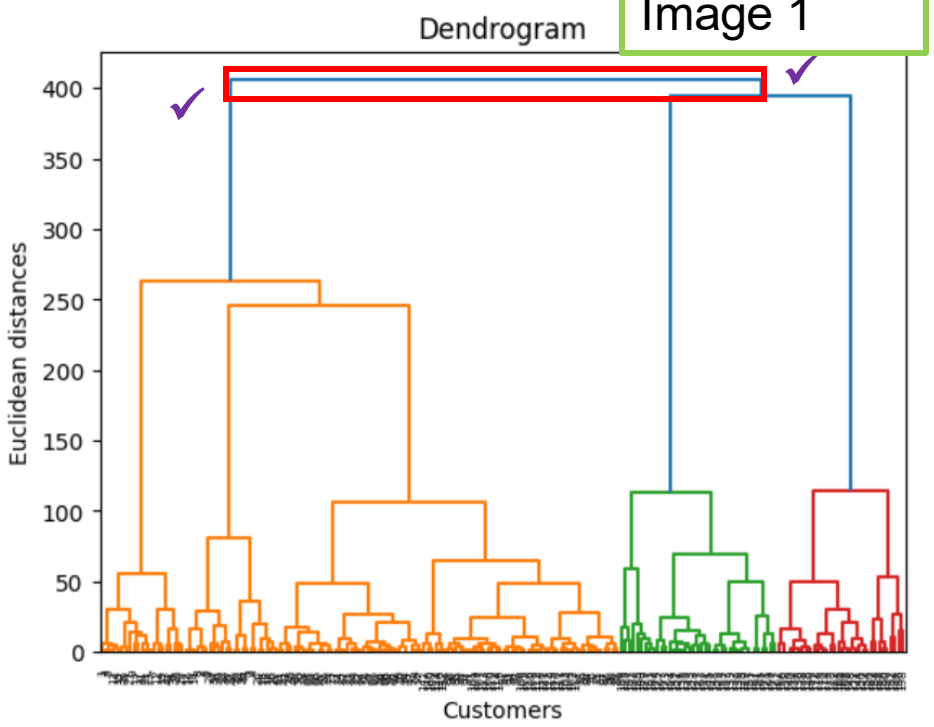
Hierarchical Clustering – Steps in Python



- Finding optimum number of clusters
  - Draw rectangular bar from top horizontal bar to next horizontal bar and count the number of vertical bars

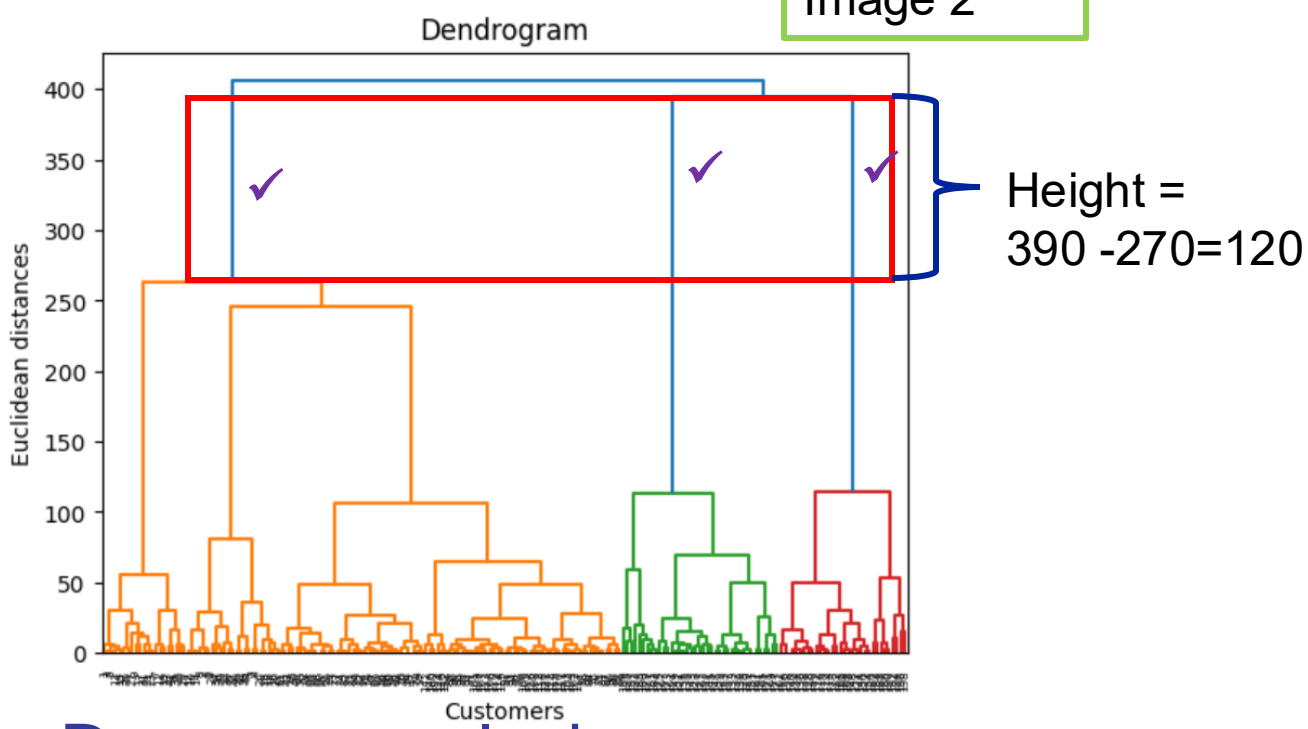
Hierarchical Clustering – Steps in Python

Image 1



Draw rectangular bar between first and second horizontal bar and count vertical bar = 2

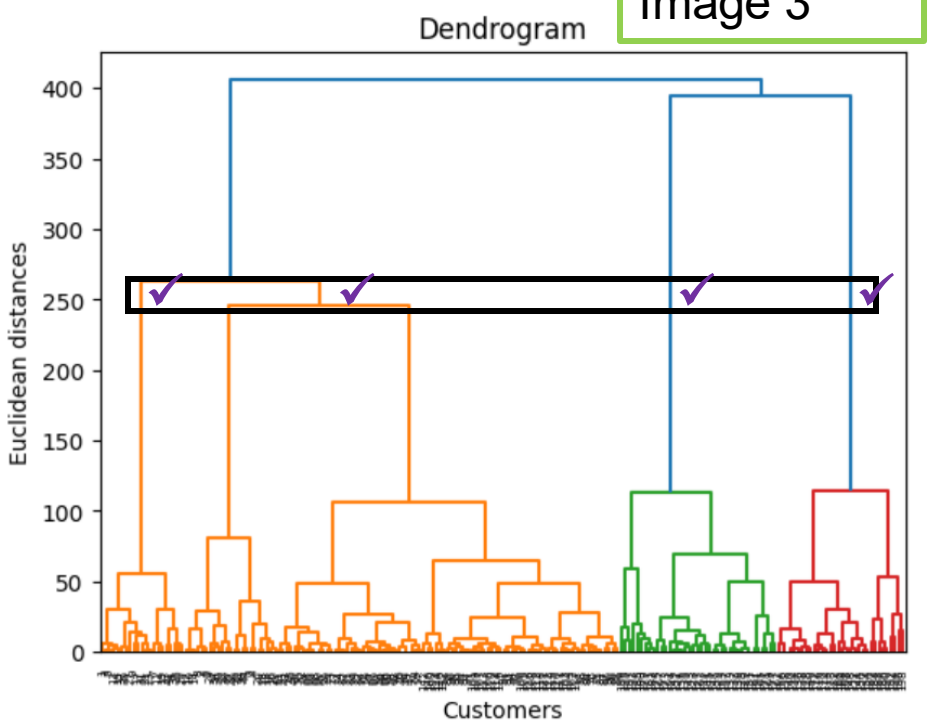
Image 2



Draw rectangular bar between second and third horizontal bar and count vertical bar = 3

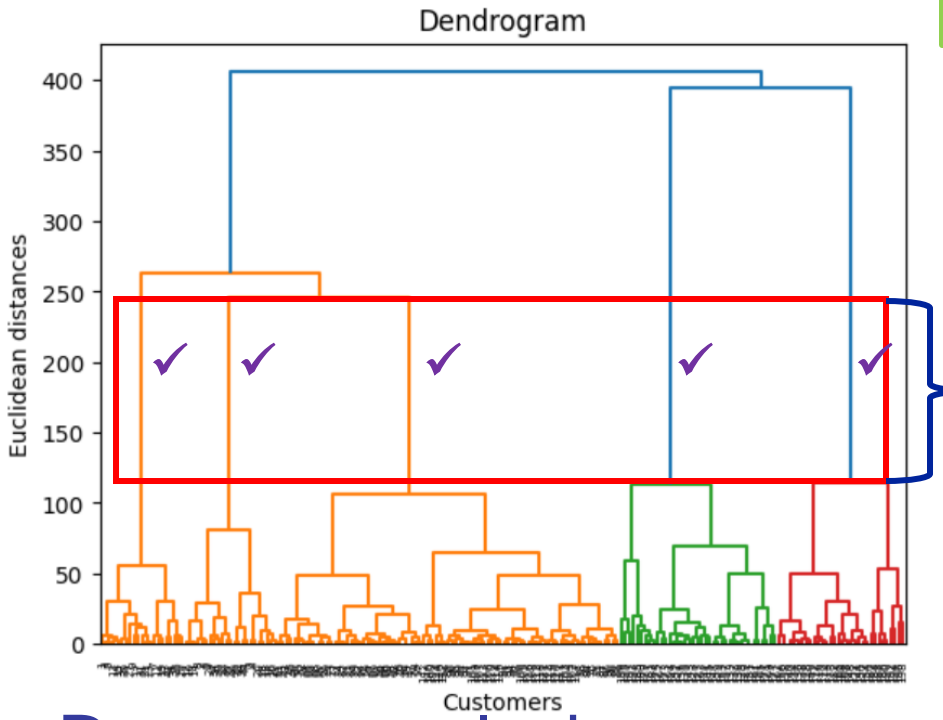
Hierarchical Clustering – Steps in Python

Image 3



Draw rectangular bar between third and fourth horizontal bar and count vertical bar = 4

Image 4



Height = 240 -110=130

Draw rectangular bar between fourth and fifth horizontal bar and count vertical bar = 5

Hierarchical Clustering – Steps in Python

- If we draw rectangular bar further, then height will not be significant like in Image 2 & Image 4. So, stop drawing the rectangle box
- Since the height of image 4 is higher than other images, so **optimal number of cluster = 5**

Hierarchical Clustering – Steps in Python

Training the Hierarchical Clustering model on the dataset

```
[4] from sklearn.cluster import AgglomerativeClustering
    hc = AgglomerativeClustering(n_clusters = 5, affinity = 'euclidean', linkage = 'ward')
    y_hc = hc.fit_predict(X)
```

Apply the clustering configuration to the variable X

Optimal number of cluster (see previous slide)

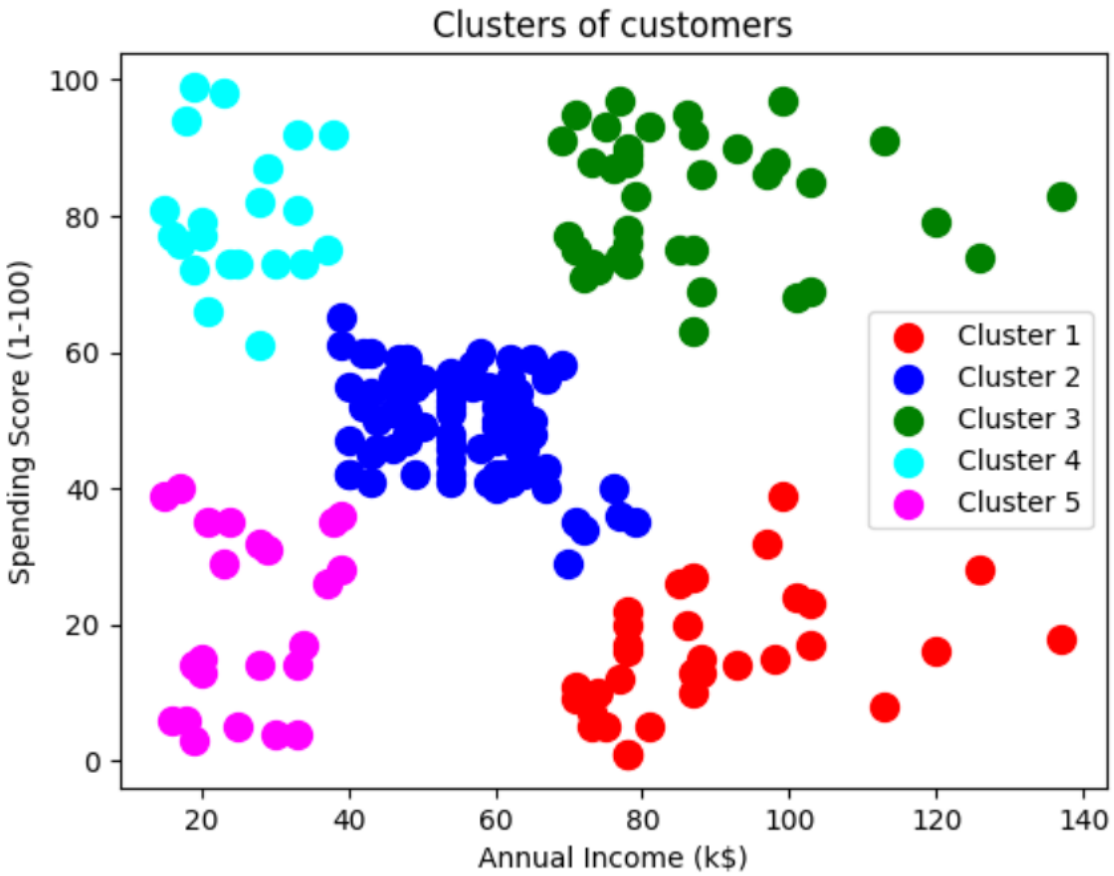
Method used to find the distance between the cluster

Hierarchical Clustering – Steps in Python

Visualising the clusters

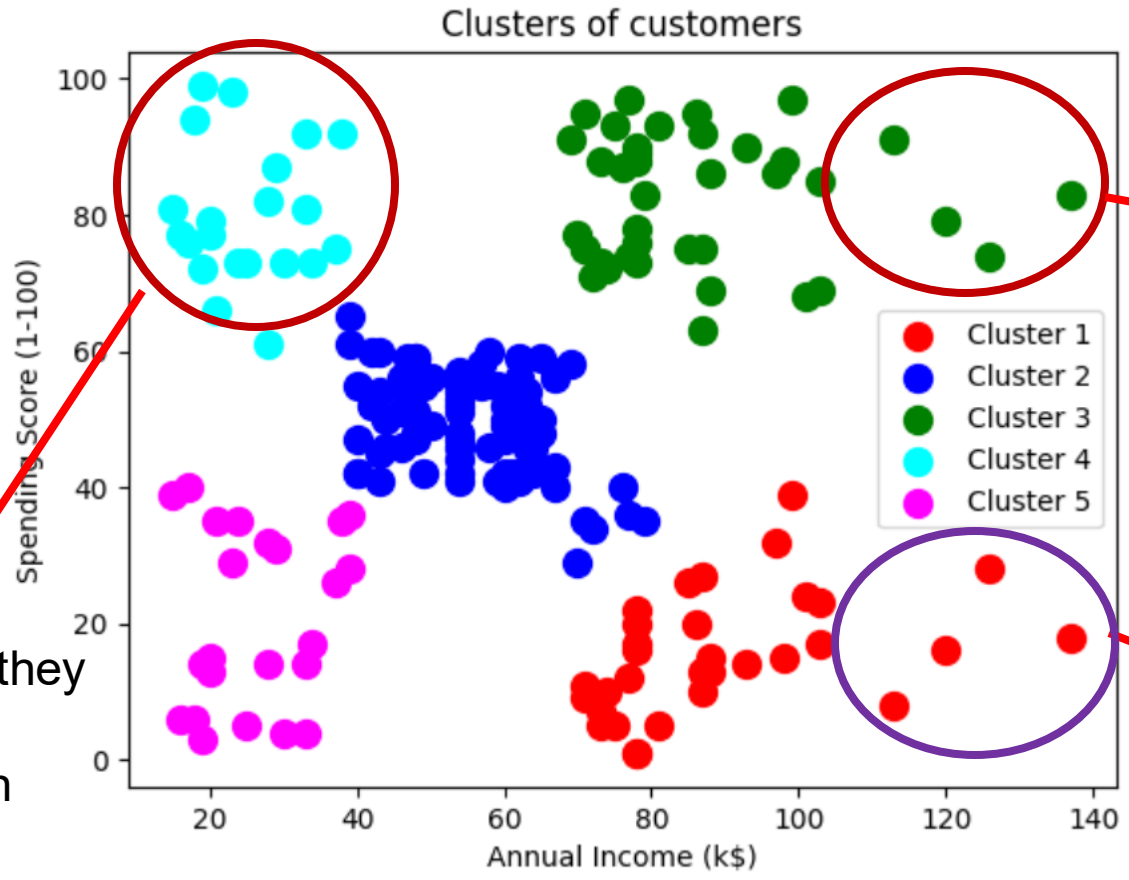
```
[5] plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
    plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s = 100, c = 'blue', label = 'Cluster 2'
    plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s = 100, c = 'green', label = 'Cluster 3
    plt.scatter(X[y_hc == 3, 0], X[y_hc == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4'
    plt.scatter(X[y_hc == 4, 0], X[y_hc == 4, 1], s = 100, c = 'magenta', label = 'Cluster
    plt.title('Clusters of customers')
    plt.xlabel('Annual Income (k$)')
    plt.ylabel('Spending Score (1-100)')
    plt.legend()
    plt.show()
```

Hierarchical Clustering – Steps in Python



Hierarchical Clustering– Steps in Python

- Sales Strategy



Target this customer since they are having high income and high spending through advertisement

Don't target this customer since they are having low income and high spending

Target this customer since they are having high income and low spending by giving them sales offer

Project

18. Perform the cluster analysis such as K-means and Hierarchical for any field from the chosen dataset.
19. Explain the strategy for improving the system after viewing the cluster diagram.

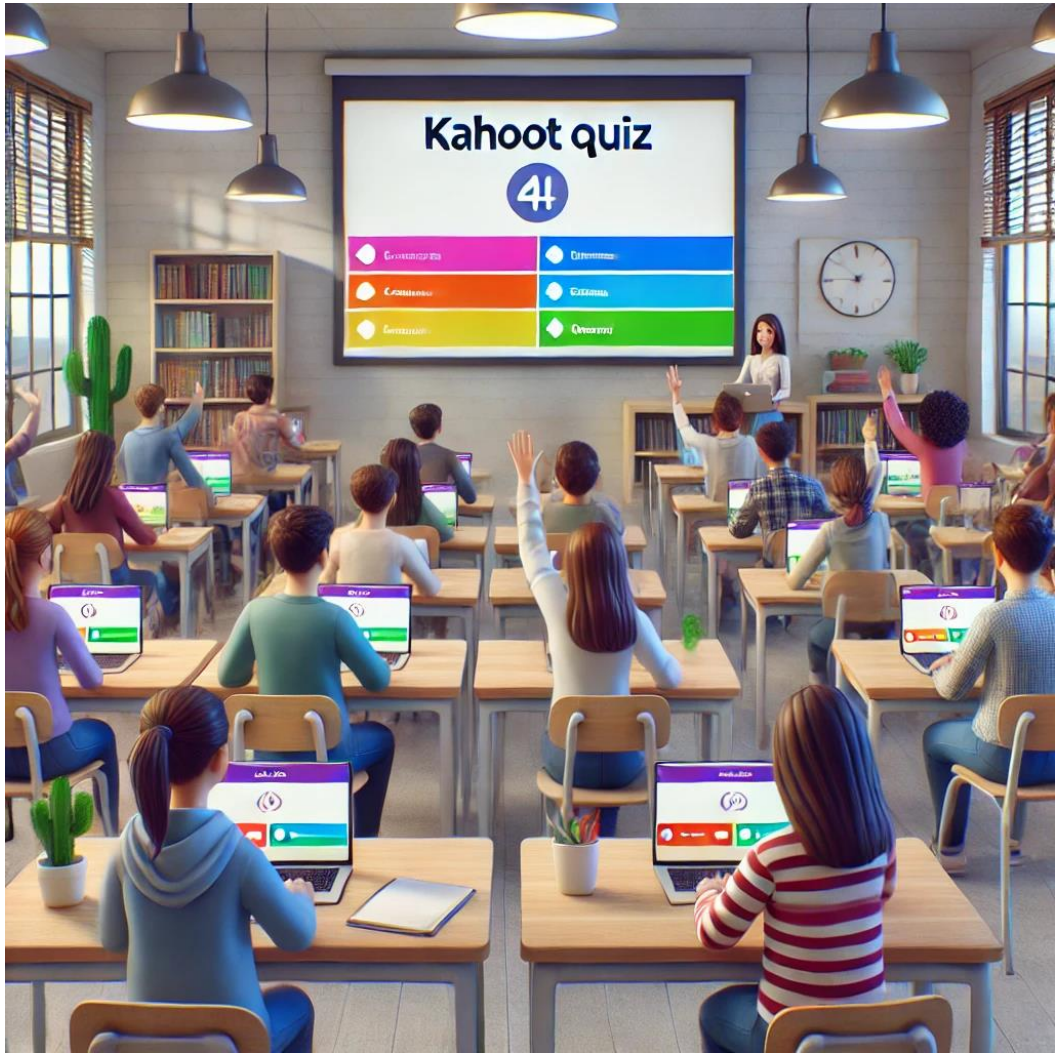
Skill Yourself – Other Cluster Analysis

Cluster Analysis Method	Description	Evaluation Methods
Agglomerative Hierarchical Clustering	A bottom-up approach where initially each data point is a separate cluster and then pairs of clusters are merged iteratively.	Dendrogram visualization, cophenetic correlation coefficient
Hierarchical Clustering	Creates a hierarchy of clusters, either in a bottom-up (agglomerative) or top-down (divisive) manner.	Dendrogram visualization, cophenetic correlation coefficient
K-Means Clustering	Partitions data into k clusters by minimizing the within-cluster sum of squares.	Silhouette Coefficient, Davies-Bouldin Index, Calinski-Harabasz Index, Elbow Method
DBSCAN (Density-Based Spatial Clustering of Applications with Noise)	Groups together data points that are closely packed together, 1 forming dense regions.	-
Gaussian Mixture Models (GMM)	Assumes that the data points are generated from a mixture of Gaussian distributions.	Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC)
Mean Shift Clustering	Performs clustering by finding the modes (peaks) in the density of data points.	-
Spectral Clustering	Transforms the data into a lower-dimensional space using spectral graph theory and then applies k-means clustering.	Silhouette Coefficient, Normalized Cut
Fuzzy C-Means	Allows data points to belong to multiple clusters with varying degrees of membership.	Partition Coefficient, Xie-Beni Index
OPTICS (Ordering Points To Identify the Clustering Structure)	Finds clusters of varying densities in the data by creating an ordered list of points.	-
Affinity Propagation	Finds exemplars (data points that represent a cluster) by sending messages between data points.	-
BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)	Efficiently clusters large datasets by creating a summary tree of the data.	Silhouette Coefficient, Davies-Bouldin Index
Self-Organizing Map (SOM)	Projects high-dimensional data onto a two-dimensional grid, preserving topological relationships.	Visualization, quantization error
K-medoids	Similar to k-means but selects data points as cluster centers instead of calculating the mean of each cluster.	Silhouette Coefficient, Davies-Bouldin Index

Conclusion

- Understanding and applying **agglomerative** and **divisive clustering** techniques enable effective data segmentation.
- Practical implementation in **Python** demonstrates its utility in real-world scenarios, such as identifying customer segments.
- **Hierarchical Clustering** is a robust method for organizing complex datasets and uncovering hidden patterns, making it essential for **data-driven decision-making**.

Kahoot Time!



Higher Colleges of Technology

كليات التقنية العليا

Thank You